IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Language and dialect recognition for cuneiform texts

SUPERVISOR : BASTIEN PASDELOUP
AUTHORS : QUENTIN ANDRÉ · MAËLLE GABENS

CONTEXT

Mesopotamia is the ancient name given to the region between the Euphrates and Tigris rivers, in what is now Iraq and parts of Turkey, Syria and Iran.
In the **3rd millennium BC**, the **cuneiform writing** was invented here and was one of the first writings of the world. It was used for about 4 millenniums.

Location of the Mesopotamian empires from 3rd millennium to 1st BC 1

The cuneiform writing system consisted of signs printed on fresh clay tablets. Clay is a material that can withstand the passage of time without suffering significant deterioration. This is why archaeologists have managed to find large quantities of these tablets in a very good state of preservation.

Cuneiform writing on clay tablet 2

Initially used to write the Sumerian language, it was later adapted to write several other languages of the region, notably Akkadian or Babylonian in different dialects. Along time, it was transformed from a purely logographic language (one sign for one object), to a **logo-syllabic** language (a word can be either a symbol or a combination of several symbols).
It makes it **hard to compute**, especially since there is no punctuation or spaces 3.

SOURCES

1 *https://whatmaster.com/mesopotamia/*

2 *Wikimedia: Cuneiform_tablet-MAHG_15856-IMG_9472-black.jpg*

3 *https://towardsdatascience.com/assyrian-or-babylonian-language-identification-in-cuneiform-texts-4f15a14a5d70#0304*

4 *Jauhiainen et al: Language and Dialect Identification of Cuneiform Texts, 2019*

5 *Dencker et al: Deep Learning of Cuneiform Sign Detection with Weak Supervision using Transliteration Alignment, 2020*

6 *Wu et al: Language Discrimination and Transfer Learning for Similar Languages: Experiments with Feature Combinations and Adaptation, 2019*

7 *Benites et al: TwistBytes - Identification of Cuneiform Languages and German Dialects at VarDial, 2019*

8 *Jauhiainen et al: HeLI, a Word-Based Backoff Method for Language Identification, 2019*

9 *Mikolov et al.: Efficient Estimation of Word Representations in Vector Space, 2013*

10 *https://medium.com/@hari4om/word-embedding-d816f643140*

11 *https://towardsdatascience.com/bert-explained-state-of-the-art-language-model-for-nlp-f8b21a9b6270*

12 *Goutte et al: Improving Cuneiform Language Identification with BERT, 2019*

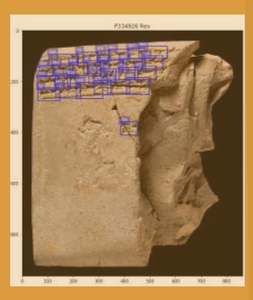
13 *Chung et al: Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling, 2014*

14 *https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21*

CUNEIFORM LANGUAGE IDENTIFICATION (CLI)

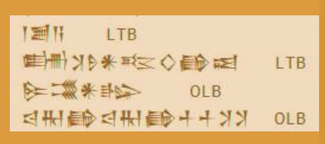
The CLI Challenge was launched by **VarDial** in 2019. Different research teams competed to identify Mesopotamian languages and dialects with machine learning 4 . The provided dataset has the following characteristics:

- **7 languages** or dialects to identify
- samples are lines of 3 to 145 cuneiform signs from about 1800 clay tablets (cf image on the right) of the *Open Richly Annotated Cuneiform Corpus* (ORACC) 5
- **Unbalanced** for training, not for validation and test
- A lot of **duplicates** (about 30%)



Language or dialect	acronym	training	validation	test
Sumerian	SUX	53673	668	985
Old Babylonian	OLB	3803	668	985
Middle Babylonian perypheral	MBP	5508	668	985
Standard Babylonian	STB	17817	668	985
Neo-Babylonina	NEB	9707	668	985
Late Babylonian	LTB	15947	668	985
Neo-Assyrian	NEA	32966	668	985

Translated into **Unicode**:
- the way of writing signs can no longer be used
- missing characters were simply removed



Some further preprocessing were used by some of the competitors, such as **data augmentation** and **line segmentation**. 6

NGRAMS

Step 1 : Language representations

- extract **every subset of n contiguous signs** of a line
- do it for the whole training and testing data 8

*ex : in **"happy"** ; List_1_gram = ["h","a","p","y"] ; List_2_gram=["ha","ap","pp","py"] ...*

USED AS INPUT

HELI

Step 2 : Estimators

- There are 2 parameters:
  - p used as a score when a sign is found in the tested line and was not present in a training set (avoids null scores)
  - [nmin,nmax], the range of the ngrams used for the score computation
- First we calculate the frequency of the n_grams for each n = 1,nmax and each language. Then we calculate the **score for every n_gram for each language.**

$$score_{ngram_i} = \begin{cases} -\log_{10}\left(\frac{nb_{ngram_i}}{nb_{tot_{ngram}}}\right) & ,if\ nb_{ngram_i} > 0 \\ p & ,if\ nb_{ngram_i} = 0 \end{cases}$$

- We define the function dg(t, n) for counting n-grams in phrase t found in a model M.

$$dg(t,n) = \sum_{i=0}^{len(t)-n} \begin{cases} 1 & ,if\ t[i:i+n] \in M \\ 0 & ,if\ t[i:i+n] \notin M \end{cases}$$

$$score(t,n) = \begin{cases} \frac{1}{dg(t,n)} \sum_{i=0}^{len(t)-n} score_{ngram_i} & ,if\ \ dg(t,n) > 0 \\ score(t,n-1) & ,if\ dg(t,n) = 0 \end{cases}$$

- the prediction for one line is its lowest score among the 7 dialects

Used as baseline for the competition, its accuracy is **0.7061.** 4

We reproduced this algorithm. The following results were obtained :

language	LTB	MPB	NEA	NEB	OLB	STB	SUX	TOTAL
accuracy, nmax = 1	0.109645	0.3086294	0.0578680	0.259898	0.00203	0.902538	0.281218	0.274547
accuracy, nmax = 2	0.395939	0.434518	0.231472	0.2649746	0.025381	0.812183	0.222335	0.340972
accuracy, nmax = 3	0.708629	0.7553299	0.337056	0.294416	0.142132	0.448731	0.260914	0.42103
accuracy, nmax = 4	0.485279	0.617259	0.215228	0.258883	0.254822	0.3147208	0.140102	0.326613

MACHINE LEARNING FOR LANGUAGE IDENTIFICATION

Step 1 : Language representations

- Make **text** usable by other algorithms: transformed **into vectors** of numbers
- **Unsupervised**: no labels or scores for the training data, so these algorithms have no need to know the language to extract the useful information of the line

Step 2 : Classifiers

- Can contain unsupervised training, but at least one **supervised** algorithm, which uses a **label** to learn how to predict if a sample belongs to a class

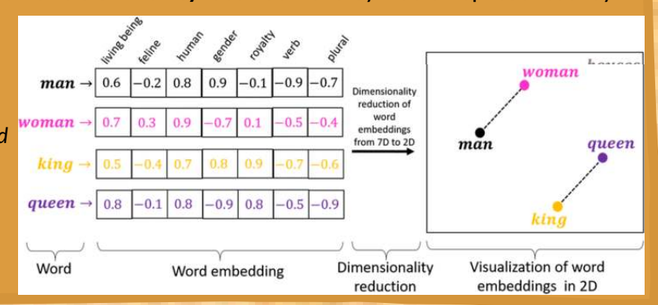
Step 3 : Ensemble learning (optional)

- **Aggregates** the results of estimators
- Performs better here than adding features in language representation 7

EMBEDDINGS (WORD2VEC)

Step 1 : Language representations

- based on a **neural network**, learns **word associations** from a large corpus of text
- represents each symbol with a vector such that the **cosine similarity between the vectors indicates the semantic similarity** between the symbols represented by those vectors 9



Example of word embedding and the link between semantic and cosine similarity 10

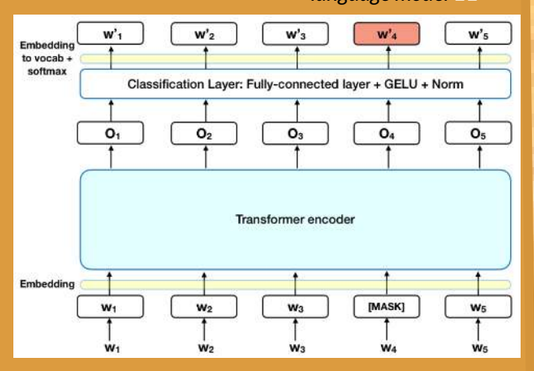
USED AS INPUT

BERT

Step 2 : Estimators

- ***Input*** **:** matrix of shape (number of symbols in the line) * (size of embedding vector)
- **Deep neural network**
- ***Layer of transformers***: takes a list of symbol embeddings as input and outputs the list of the **same vectors transformed** to take the **context** into account, and **the symbols are weighted by their significance** in the output
- ***Final layer*** **:** fully connected layer for classification

Process of the masked language model 11



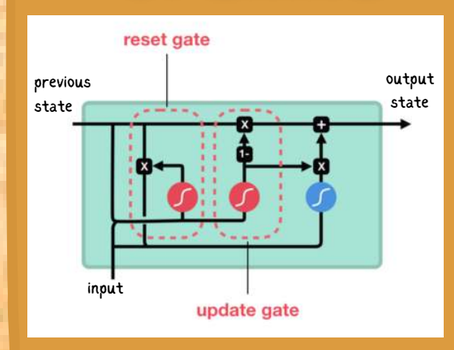
Trained with several steps:
- Unsupervised **masked language model**: mask a sign of the input line, and fit the weights of the neural network to predict the masked sign from the context (the other signs of the line)
- Supervised **sentence pair classification**: predicts if two lines come from the same language
- **Fine tuning** for final classifier

System	LTB	MPB	NEA	NEB	OLB	STB	SUX
char[1-4]	0.9317	0.8296	**0.7402**	0.6317	0.7681	0.5345	0.7539
Ensemble	0.9339	0.8339	0.7336	0.6316	0.7774	0.5344	0.7692
BERT	**0.9565**	**0.8666**	0.7103	**0.6500**	**0.8373**	**0.5705**	**0.7952**

Results achieved by the NRC-CNRC team 5

BERT achieved the **best** results of the competition with an accuracy of **0.7695**. This algorithm is now **widely used** in a lot of language applications such as language identification, like here, but also word prediction, translation... thanks to its adaptability through **transfer learning** (reuse already trained layers and train only the last ones) and efficiency, in term of results and computational cost. 12

OPENING



Structure of one GRU neuron 14

We tried to implement another algorithm than the ones presented in the competition: **GRU** (Gated Recurrent Unit) based on **embeddings**. 13

- ***Input*** : matrix of shape (number of symbols, padded at the beginning of the line) * (size of embedding vector)
- ***Layer of GRU neurons***:
  - **recurrent layer**: for each neuron, a **state** matrix is added to keep the information of the previous step. An update and reset gate are fitted to decide what **ratio of new information** from the next symbol of the line should be used to update the state.
  - solves the **vanishing gradient** problem of recurrent neural network (ie nothing is learning when reaching too many iterations)
- ***Final layer*** : fully connected layer for classification

We obtained an accuracy of **0.44**, which ranks us 8th out of 9, and with a much longer computational time (BERT avoids it by avoiding the recurrence, thanks to the transformers).